

Introducción a la Programación con Python

Semana 9: Conceptos Avanzados

∇Nablabs

Contenido

- 1 Conjuntos (sets)
- 2 Diccionarios avanzados
- 3 Funciones lambda
- 4 List comprehensions
- 5 Dictionary comprehensions
- 6 *args y **kwargs
- 7 Generadores e iteradores
- 8 Decoradores básicos
- 9 Manejo de APIs simples
- 10 Entornos virtuales
- 11 Proyecto integrador

¿Qué son los sets?

- Colección **desordenada** de elementos únicos
- ✓ No permite duplicados (los elimina automáticamente)
- ✗ Sin índice ni orden garantizado
- 🔑 Operaciones muy rápidas (membership, unión, intersección)
- ✍ Mutable (se puede modificar)

💡 ¡Perfecto para eliminar duplicados!

Crear sets +

```
1 # Formas de crear sets
2 numeros = {1, 2, 3, 4, 5}
3 letras = set(['a', 'b', 'c', 'a']) # Elimina 'a' duplicada
4
5 # Set vacío (¡OJO! {} crea dict, no set)
6 vacio = set()
7
8 # Desde string (elimina duplicados)
9 caracteres = set("hola")
10
11 print(numeros)
12 print(letras)
13 print(caracteres)
```

➔ Salida

```
{1, 2, 3, 4, 5}
{'a', 'b', 'c'}
{'h', 'o', 'l', 'a'}
```

! El orden de los elementos puede variar

Operaciones con sets

```
1 frutas = {'manzana', 'pera', 'uva'}
2
3 # Agregar
4 frutas.add('kiwi')
5 print(f"Agregar: {frutas}")
6
7 # Eliminar (error si no existe)
8 frutas.remove('pera')
9 print(f"Remove: {frutas}")
10
11 # Eliminar (sin error si no existe)
12 frutas.discard('banana')
13
14 # Limpiar
15 frutas_copia = frutas.copy()
16 frutas_copia.clear()
17 print(f"Clear: {frutas_copia}")
```

➡ Salida

```
Agregar: {'manzana', 'pera', 'uva', 'kiwi'}
Remove: {'manzana', 'uva', 'kiwi'}
Clear: set()
```

Operaciones matemáticas de sets

```
1 a = {1, 2, 3, 4}
2 b = {3, 4, 5, 6}
3
4 # Unión (todos los elementos)
5 print(f"Unión: {a | b}")
6 print(f"Union: {a.union(b)}")
7
8 # Intersección (elementos comunes)
9 print(f"Intersección: {a & b}")
10 print(f"Intersection: {a.intersection(b)}")
11
12 # Diferencia (en a pero no en b)
13 print(f"Diferencia: {a - b}")
14
15 # Diferencia simétrica (no comunes)
16 print(f"Dif. simétrica: {a ^ b}")
```

➡ Salida

Unión: {1, 2, 3, 4, 5, 6}

Union: {1, 2, 3, 4, 5, 6}

Intersección: {3, 4}

Intersection: {3, 4}

Diferencia: {1, 2}

Dif. simétrica: {1, 2, 5, 6}

💡 Los operadores `—` & `-` `^` son más legibles que los métodos

Sets: Caso de uso práctico

```
1 # Eliminar duplicados de una lista
2 numeros = [1, 2, 2, 3, 4, 4, 4, 5, 1, 2]
3 unicos = list(set(numeros))
4 print(f"Original: {numeros}")
5 print(f"Sin duplicados: {unicos}")
6
7 # Encontrar elementos únicos en dos listas
8 lista1 = ['python', 'java', 'c++', 'python']
9 lista2 = ['javascript', 'python', 'ruby']
10
11 comunes = set(lista1) & set(lista2)
12 solo_lista1 = set(lista1) - set(lista2)
13
14 print(f"Comunes: {comunes}")
15 print(f"Solo en lista1: {solo_lista1}")
```


Métodos útiles

Más allá de lo básico: métodos avanzados para trabajar con diccionarios

```
1 persona = {'nombre': 'Ana', 'edad': 25, 'ciudad': 'Madrid'}
2
3 # get() con valor por defecto
4 print(persona.get('pais', 'No especificado'))
5
6 # setdefault() - obtiene o crea con valor default
7 persona.setdefault('profesion', 'Estudiante')
8 print(persona)
9
10 # update() - actualizar múltiples valores
11 persona.update({'edad': 26, 'email': 'ana@ejemplo.com'})
12 print(persona)
```

➡ Salida

No especificado

```
{ 'nombre': 'Ana', 'edad': 25, 'ciudad': 'Madrid', 'profesion': 'Estudiante' }  
{ 'nombre': 'Ana', 'edad': 26, 'ciudad': 'Madrid', 'profesion': 'Estudiante', 'email':  
'ana@ejemplo.com' }
```

```
1 # Diccionarios dentro de diccionarios
2 empresa = {
3     'empleados': {
4         'E001': {'nombre': 'Ana', 'salario': 50000},
5         'E002': {'nombre': 'Luis', 'salario': 60000}
6     },
7     'departamentos': {
8         'IT': ['E001'],
9         'Ventas': ['E002']
10    }
11 }
12
13 # Acceder a datos anidados
14 print(empresa['empleados']['E001']['nombre'])
15 print(empresa['departamentos']['IT'])
16
17 # Iterar sobre diccionario anidado
18 for id_emp, datos in empresa['empleados'].items():
19     print(f"{id_emp}: {datos['nombre']} - ${datos['salario']}")
```

➡ Salida

```
Ana  
E001',
```

```
E001: Ana - $50000
```

```
E002: Luis - $60000
```

¿Qué son?

Funciones **anónimas** de una sola expresión. Sintaxis compacta para funciones simples.

Sintaxis:

```
1 lambda argumentos: expresion
```

Comparación:

```
1 # Función tradicional
2 def sumar(a, b):
3     return a + b
4
5 # Función lambda equivalente
6 sumar_lambda = lambda a, b: a + b
7
8 print(sumar(3, 5))
9 print(sumar_lambda(3, 5))
```

Lambda con map(), filter(), sorted()

```
1 numeros = [1, 2, 3, 4, 5]
2
3 # map() - aplicar función a cada elemento
4 cuadrados = list(map(lambda x: x**2, numeros))
5 print(f"Cuadrados: {cuadrados}")
6
7 # filter() - filtrar elementos
8 pares = list(filter(lambda x: x % 2 == 0, numeros))
9 print(f"Pares: {pares}")
10
11 # sorted() con key personalizada
12 palabras = ['python', 'es', 'genial', 'y', 'poderoso']
13 ordenadas = sorted(palabras, key=lambda x: len(x))
14 print(f"Por longitud: {ordenadas}")
15
16 personas = [{'nombre': 'Ana', 'edad': 25}, {'nombre': 'Luis', '
    edad': 20}]
17 por_edad = sorted(personas, key=lambda p: p['edad'])
18 print(f"Por edad: {por_edad}")
```

➔ Salida

Cuadrados: [1, 4, 9, 16, 25]

Pares: [2, 4]

Por longitud: ['y', 'es', 'genial', 'python', 'poderoso']

Por edad: [{'nombre': 'Luis', 'edad': 20}, {'nombre': 'Ana', 'edad': 25}]

i Lambda es útil para funciones simples y temporales

List comprehensions ☰

¿Qué son?

Forma concisa y elegante de crear listas a partir de iterables. Más rápidas y legibles que bucles tradicionales.

Sintaxis:

```
1 [expresion for elemento in iterable if condicion]
```

Comparación:

```
1 # Forma tradicional
2 cuadrados = []
3 for x in range(5):
4     cuadrados.append(x**2)
5
6 # List comprehension
7 cuadrados = [x**2 for x in range(5)]
8
9 print(cuadrados)
```


List comprehensions con condiciones



```
1 # Números pares
2 pares = [x for x in range(10) if x % 2 == 0]
3 print(f"Pares: {pares}")
4
5 # Con transformación y condición
6 cuadrados_impares = [x**2 for x in range(10) if x % 2 != 0]
7 print(f"Cuadrados impares: {cuadrados_impares}")
8
9 # Múltiples condiciones
10 divisibles = [x for x in range(20) if x % 2 == 0 if x % 3 == 0]
11 print(f"Divisibles por 2 y 3: {divisibles}")
12
13 # If-else (diferente sintaxis)
14 resultado = ['par' if x % 2 == 0 else 'impar' for x in range(5)]
15 print(f"Par/Impar: {resultado}")
```

➡ Salida

Pares: [0, 2, 4, 6, 8]

Cuadrados impares: [1, 9, 25, 49, 81]

Divisibles por 2 y 3: [0, 6, 12, 18]

Par/Impar: ['par', 'impar', 'par', 'impar', 'par']

List comprehensions anidadas

```
1 # Matriz (lista de listas)
2 matriz = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
3
4 # Aplanar matriz
5 plana = [num for fila in matriz for num in fila]
6 print(f"Aplanada: {plana}")
7
8 # Crear matriz con comprehension
9 matriz_nueva = [[i*j for j in range(3)] for i in range(3)]
10 print(f"Matriz nueva: {matriz_nueva}")
11
12 # Combinar dos listas
13 colores = ['rojo', 'verde', 'azul']
14 objetos = ['coche', 'casa']
15 combinaciones = [f"{color} {objeto}"
16                  for color in colores
17                  for objeto in objetos]
18 print(f"Combinaciones: {combinaciones}")
```

➡ Salida

Aplanada: [1, 2, 3, 4, 5, 6, 7, 8, 9]

Matriz nueva: [[0, 0, 0], [0, 1, 2], [0, 2, 4]]

Combinaciones: ['rojo coche', 'rojo casa', 'verde coche', 'verde casa', 'azul coche', 'azul casa']

Dictionary comprehensions

¿Qué son?

Similar a list comprehensions pero para crear diccionarios de forma concisa.

Sintaxis:

```
1 {clave: valor for elemento in iterable if condicion}
```

```
1 # Crear diccionario de cuadrados
2 cuadrados = {x: x**2 for x in range(5)}
3 print(cuadrados)
4
5 # Desde dos listas
6 claves = ['nombre', 'edad', 'ciudad']
7 valores = ['Ana', 25, 'Madrid']
8 persona = {k: v for k, v in zip(claves, valores)}
9 print(persona)
```

➔ Salida

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Dict comprehensions con condiciones



```
1 # Filtrar diccionario
2 notas = {'Ana': 85, 'Luis': 92, 'María': 78, 'Pedro': 95}
3
4 # Solo aprobados (>= 80)
5 aprobados = {nombre: nota for nombre, nota in notas.items()
6               if nota >= 80}
7 print(f"Aprobados: {aprobados}")
8
9 # Transformar valores
10 incrementadas = {nombre: nota + 5 for nombre, nota in notas.
11                  items()}
12 print(f"Incrementadas: {incrementadas}")
13
14 # Invertir diccionario
15 invertido = {v: k for k, v in notas.items()}
16 print(f"Invertido: {invertido}")
```

Salida

```
Aprobados: {'Ana': 85, 'Luis': 92, 'Pedro': 95}
```

```
Incrementadas: {'Ana': 90, 'Luis': 97, 'María': 83, 'Pedro':  
100}
```

```
Invertido: {85: 'Ana', 92: 'Luis', 78: 'María', 95: 'Pedro'}
```

Argumentos variables: *args ...

¿Qué es *args?

Permite pasar un número **variable** de argumentos posicionales a una función. Se reciben como tupla.

```
1 def sumar(*args):
2     """Suma cualquier cantidad de números"""
3     total = sum(args)
4     print(f"Argumentos: {args}")
5     return total
6
7 print(sumar(1, 2, 3))
8 print(sumar(10, 20, 30, 40, 50))
9
10 def info(nombre, *otros_datos):
11     print(f"Nombre: {nombre}")
12     print(f"Otros datos: {otros_datos}")
13
14 info("Ana", 25, "Madrid", "Ingeniera")
```


➔ Salida

```
Argumentos: (1, 2, 3)
```

```
6
```

```
Argumentos: (10, 20, 30, 40, 50)
```

```
150
```

```
Nombre: Ana
```

```
Otros datos: (25, 'Madrid', 'Ingeniera')
```

i El nombre 'args' es convención, puedes usar cualquier nombre

Argumentos variables: `**kwargs`

¿Qué es `**kwargs`?

Permite pasar un número **variable** de argumentos con nombre (keyword).
Se reciben como diccionario.

```
1 def crear_perfil(**kwargs):  
2     """Crea perfil con datos variables"""  
3     print("Perfil creado:")  
4     for clave, valor in kwargs.items():  
5         print(f"  {clave}: {valor}")  
6     return kwargs  
7  
8 crear_perfil(nombre="Ana", edad=25, ciudad="Madrid")  
9 print()  
10 crear_perfil(nombre="Luis", profesion="Ingeniero", email="luis@ejemplo.com")
```

➡ Salida

Perfil creado:

nombre: Ana

edad: 25

ciudad: Madrid

Perfil creado:

nombre: Luis

profesion: Ingeniero

email: luis@ejemplo.com

Combinando *args y **kwargs

```
1 def funcion_completa(arg1, arg2, *args, kwarg1="default", **
  kwargs):
2     print(f"arg1: {arg1}")
3     print(f"arg2: {arg2}")
4     print(f"args: {args}")
5     print(f"kwarg1: {kwarg1}")
6     print(f"kwargs: {kwargs}")
7
8 funcion_completa(1, 2, 3, 4, kwarg1="personalizado",
9                 extra1="valor1", extra2="valor2")
```

➔ Salida

```
arg1: 1
arg2: 2
args: (3, 4)
kwarg1: personalizado
kwargs: {'extra1': 'valor1', 'extra2': 'valor2'}
```

¿Qué son?

Funciones que generan valores **uno a la vez** usando `yield`. Más eficientes en memoria que listas.

```
1 # Función generadora
2 def contador(maximo):
3     n = 0
4     while n < maximo:
5         yield n # Retorna valor y pausa
6         n += 1
7
8 # Usar generador
9 gen = contador(5)
10 print(type(gen))
11
12 for numero in gen:
13     print(numero, end=' ')
```

➔ Salida

```
<class 'generator'>
```

Generadores vs Listas

```
1 import sys
2
3 # Lista (toda en memoria)
4 lista = [x**2 for x in range(1000)]
5 print(f"Tamaño lista: {sys.getsizeof(lista)} bytes")
6
7 # Generador (valores bajo demanda)
8 generador = (x**2 for x in range(1000))
9 print(f"Tamaño generador: {sys.getsizeof(generador)} bytes")
10
11 # Usar generador
12 primeros_5 = [next(generador) for _ in range(5)]
13 print(f"Primeros 5: {primeros_5}")
```

➔ Salida

```
Tamaño lista: 8856 bytes
Tamaño generador: 104 bytes
Primeros 5: [0, 1, 4, 9, 16]
```

💡 Generadores son ideales para grandes volúmenes de datos

Generador: Fibonacci ∞

```
1 def fibonacci(limite):
2     """Genera números de Fibonacci hasta un límite"""
3     a, b = 0, 1
4     while a < limite:
5         yield a
6         a, b = b, a + b
7
8 # Generar Fibonacci hasta 100
9 for num in fibonacci(100):
10     print(num, end=' ')
11 print()
12
13 # Convertir a lista
14 fib_lista = list(fibonacci(50))
15 print(f"\nFibonacci < 50: {fib_lista}")
```

➡ Salida

0 1 1 2 3 5 8 13 21 34 55 89

Fibonacci < 50: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

¿Qué son?

Funciones que modifican o extienden el comportamiento de otras funciones sin cambiar su código.

```
1 def mi_decorador(funcion):
2     def wrapper():
3         print("Antes de la función")
4         funcion()
5         print("Después de la función")
6     return wrapper
7
8 @mi_decorador
9 def saludar():
10     print("¡Hola!")
11
12 saludar()
```

➔ Salida

```
Antes de la función
¡Hola!
```


Decorador con argumentos

```
1 def decorador_con_args(funcion):
2     def wrapper(*args, **kwargs):
3         print(f"Llamando a {funcion.__name__}")
4         resultado = funcion(*args, **kwargs)
5         print(f"Resultado: {resultado}")
6         return resultado
7     return wrapper
8
9 @decorador_con_args
10 def multiplicar(a, b):
11     return a * b
12
13 @decorador_con_args
14 def sumar(a, b, c):
15     return a + b + c
16
17 multiplicar(3, 4)
18 print()
19 sumar(1, 2, 3)
```

➔ Salida

Llamando a multiplicar

Resultado: 12

Llamando a sumar

Resultado: 6

Decorador práctico: Medir tiempo

```
1 import time
2
3 def medir_tiempo(funcion):
4     def wrapper(*args, **kwargs):
5         inicio = time.time()
6         resultado = funcion(*args, **kwargs)
7         fin = time.time()
8         print(f"{funcion.__name__} tardó {fin - inicio:.4f}
          segundos")
9         return resultado
10    return wrapper
11
12 @medir_tiempo
13 def calcular_suma(n):
14     return sum(range(n))
15
16 @medir_tiempo
17 def calcular_cuadrados(n):
18     return [i**2 for i in range(n)]
19
20 calcular_suma(1000000)
21 calcular_cuadrados(10000)
```

¿Qué es una API?

Interfaz que permite comunicación entre aplicaciones. REST API usa HTTP para intercambiar datos (generalmente JSON).

Instalar requests:

```
pip install requests
```

Ejemplo básico:

```
1 import requests
2
3 # GET request a una API pública
4 url = "https://jsonplaceholder.typicode.com/posts/1"
5 respuesta = requests.get(url)
6
7 print(f"Status: {respuesta.status_code}")
8 print(f"Tipo: {type(respuesta.json())}")
9 print(f"Datos: {respuesta.json()}")
```

➡ Salida

Status: 200

Tipo: <class 'dict'>

Datos: {'userId': 1, 'id': 1, 'title': 'sunt aut...', 'body': 'quia et...'}

i Status 200 significa éxito, 404 no encontrado, 500 error del servidor

API: Obtener múltiples recursos

```
1 import requests
2
3 # Obtener lista de posts
4 url = "https://jsonplaceholder.typicode.com/posts"
5 respuesta = requests.get(url)
6
7 if respuesta.status_code == 200:
8     posts = respuesta.json()
9     print(f"Total de posts: {len(posts)}")
10
11     # Mostrar primeros 3
12     for post in posts[:3]:
13         print(f"\nID: {post['id']}")
14         print(f"Título: {post['title']}")
15         print(f"Usuario: {post['userId']}")
16 else:
17     print(f"Error: {respuesta.status_code}")
```

API: POST request

```
1 import requests
2
3 url = "https://jsonplaceholder.typicode.com/posts"
4
5 # Datos a enviar
6 nuevo_post = {
7     "title": "Mi nuevo post",
8     "body": "Este es el contenido del post",
9     "userId": 1
10 }
11
12 # Enviar POST
13 respuesta = requests.post(url, json=nuevo_post)
14
15 print(f"Status: {respuesta.status_code}")
16 print(f"Respuesta: {respuesta.json()}")
```

➔ Salida

Status: 201

Respuesta: {'title': 'Mi nuevo post', 'body': '...', 'userId': 1, 'id': 101}

¿Qué son?

Espacios aislados para cada proyecto con sus propias dependencias y versiones de paquetes.

¿Por qué usarlos?

- ✓ Evitar conflictos entre proyectos
- ✓ Mantener dependencias específicas por proyecto
- ✓ Facilitar colaboración y despliegue
- ✓ No contaminar instalación global de Python

Crear y usar entornos virtuales >_

Crear entorno virtual:

```
# Python 3
python -m venv mi_entorno

# O con virtualenv
virtualenv mi_entorno
```

Activar entorno:

```
# Linux/Mac
source mi_entorno/bin/activate

# Windows
mi_entorno\Scripts\activate
```

Desactivar:

```
deactivate
```

Instalar paquetes:

```
# Con entorno activado  
pip install requests pandas numpy
```

Guardar dependencias:

```
pip freeze > requirements.txt
```

Instalar desde requirements.txt:

```
pip install -r requirements.txt
```

💡 requirements.txt facilita compartir y replicar entornos

Ejemplo de requirements.txt

```
requests==2.31.0  
pandas==2.0.3  
numpy==1.24.3  
matplotlib==3.7.2  
pytest==7.4.0
```

Buenas prácticas:

- ✓ Un entorno virtual por proyecto
- ✓ Incluir requirements.txt en control de versiones
- ✓ Documentar versión de Python requerida
- ✓ Actualizar requirements.txt regularmente
- ✓ No incluir carpeta del entorno en git (usar .gitignore)

Objetivo

Crear una aplicación que consuma una API, procese datos y genere estadísticas usando conceptos avanzados.

Funcionalidades:

-  Consumir API REST pública
-  Filtrar y transformar datos con comprehensions
-  Calcular estadísticas usando lambda y funciones
-  Guardar resultados en archivo
-  Medir tiempos de ejecución con decoradores
-  Usar generadores para eficiencia

 Integra todos los conceptos de la semana

Proyecto: Decoradores y utilidades

```
1 import time
2 import json
3
4 def medir_tiempo(func):
5     """Decorador para medir tiempo de ejecución"""
6     def wrapper(*args, **kwargs):
7         inicio = time.time()
8         resultado = func(*args, **kwargs)
9         fin = time.time()
10        print(f"    {func.__name__} tardó {fin - inicio:.2f}s")
11        return resultado
12    return wrapper
13
14 def guardar_json(func):
15     """Decorador para guardar resultados en JSON"""
16     def wrapper(*args, **kwargs):
17         resultado = func(*args, **kwargs)
18         nombre_archivo = f"{func.__name__}_resultado.json"
19         with open(nombre_archivo, 'w') as f:
20             json.dump(resultado, f, indent=2)
21         print(f"    Guardado en {nombre_archivo}")
22         return resultado
23    return wrapper
```

Proyecto: Clase principal </>

```
1 import requests
2
3 class AnalizadorAPI:
4     def __init__(self, base_url):
5         self.base_url = base_url
6         self.datos = []
7
8     @medir_tiempo
9     def obtener_datos(self, endpoint):
10         """Obtiene datos de la API"""
11         url = f"{self.base_url}/{endpoint}"
12         respuesta = requests.get(url)
13
14         if respuesta.status_code == 200:
15             self.datos = respuesta.json()
16             print(f"    Obtenidos {len(self.datos)} registros")
17             return self.datos
18         else:
19             print(f"    Error: {respuesta.status_code}")
20             return []
```

(Continuación en siguiente slide...)

Proyecto: Filtros y análisis

```
1 def filtrar_por_usuario(self, user_id):
2     """Filtra datos por ID de usuario"""
3     return [item for item in self.datos
4             if item.get('userId') == user_id]
5
6 def generar_estadisticas(self):
7     """Genera estadísticas de los datos"""
8     if not self.datos:
9         return {}
10
11     # Usar set para usuarios únicos
12     usuarios = {item['userId'] for item in self.datos}
13
14     # Dict comprehension para contar por usuario
15     posts_por_usuario = {
16         user_id: len([p for p in self.datos if p['userId']
17 == user_id])
18         for user_id in usuarios
19     }
20
21     return {
22         'total_registros': len(self.datos),
23         'usuarios_unicos': len(usuarios),
24         'posts_por_usuario': posts_por_usuario
```

Proyecto: Generador de títulos

```
1 def generar_titulos(self, limite=None):
2     """Generador que produce títulos uno a uno"""
3     contador = 0
4     for item in self.datos:
5         if limite and contador >= limite:
6             break
7         yield item.get('title', 'Sin título')
8         contador += 1
9
10 def buscar_palabra_en_titulos(self, palabra):
11     """Busca palabra en títulos usando lambda"""
12     # Filter con lambda
13     coincidencias = list(filter(
14         lambda item: palabra.lower() in item.get('title', '')
15         ).lower(),
16         self.datos
17     ))
18     return coincidencias
```


Proyecto: Métodos de ordenamiento ⇅

```
1 def top_usuarios(self, n=5):
2     """Top N usuarios con más posts"""
3     stats = self.generar_estadisticas()
4     posts = stats['posts_por_usuario']
5
6     # Sorted con lambda para ordenar por valor
7     top = sorted(posts.items(),
8                  key=lambda x: x[1],
9                  reverse=True)[:n]
10
11     return dict(top)
12
13 @guardar_json
14 def generar_reporte(self):
15     """Genera reporte completo"""
16     return {
17         'estadisticas': self.generar_estadisticas(),
18         'top_usuarios': self.top_usuarios(3),
19         'total_posts': len(self.datos)
20     }
```

Proyecto: Uso del analizador >_

```
1 # Crear analizador
2 analizador = AnalizadorAPI("https://jsonplaceholder.typicode.com
   ")
3
4 # Obtener datos
5 analizador.obtener_datos("posts")
6
7 # Estadísticas generales
8 stats = analizador.generar_estadisticas()
9 print(f"\n    Estadísticas:")
10 print(f"    Total registros: {stats['total_registros']}")
11 print(f"    Usuarios únicos: {stats['usuarios_unicos']}")
12
13 # Top usuarios
14 print(f"\n    Top 3 usuarios:")
15 top = analizador.top_usuarios(3)
16 for user_id, count in top.items():
17     print(f"    Usuario {user_id}: {count} posts")
```

(Continuación en siguiente slide...)

```
1 # Usar generador de títulos
2 print(f"\n    Primeros 5 títulos:")
3 for i, titulo in enumerate(analizador.generar_titulos(5), 1):
4     print(f"    {i}. {titulo[:50]}...")
5
6 # Buscar palabra
7 print(f"\n    Posts sobre 'qui':")
8 resultados = analizador.buscar_palabra_en_titulos("qui")
9 print(f"    Encontrados: {len(resultados)}")
10 for post in resultados[:3]:
11     print(f"    - {post['title'][:40]}...")
12
13 # Generar reporte (usa decorador @guardar_json)
14 print(f"\n    Generando reporte...")
15 analizador.generar_reporte()
```

➡ Salida

```
obtener_datos tardó 0.45s
Obtenidos 100 registros

Estadísticas:
Total registros: 100
Usuarios únicos: 10

Top 3 usuarios:
Usuario 1: 10 posts
Usuario 2: 10 posts
Usuario 3: 10 posts

Primeros 5 títulos:
1. sunt aut facere repellat provident occaecati...
2. qui est esse...

Posts sobre 'qui':
Encontrados: 15

Generando reporte...
Guardado en generar_reporte_resultado.json
```

Alternativa

Sistema de gestión de tareas con características avanzadas

Características:

 CRUD de tareas con comprehensions

 Filtros múltiples con lambda

 Sistema de etiquetas con sets

 Estadísticas y reportes

 Persistencia en JSON

 Tracking de tiempo con decoradores

 Generador de tareas pendientes

 Recordatorios y notificaciones

Otra alternativa

Extractor de datos de sitios web con análisis

Características:

-  Scraping con requests y BeautifulSoup
-  Limpieza de datos con comprehensions
-  Almacenamiento en estructuras eficientes
-  Análisis estadístico
-  Exportación a múltiples formatos
-  Monitoreo de rendimiento
-  Procesamiento lazy con generadores

Recomendaciones

- ✓ Usa comprehensions para código más pythonic
- ✓ Prefiere generadores para grandes datasets
- ✓ Lambda solo para funciones muy simples
- ✓ Decoradores para separar concerns (logging, timing, etc.)
- ✓ Sets para operaciones de conjunto y eliminar duplicados
- ✓ Entornos virtuales siempre, uno por proyecto
- ✓ Maneja errores en requests (try-except, status codes)
- ✓ Documenta código complejo
- ✓ Type hints para código más claro
- ✓ No abusos de las características avanzadas

Type hints (bonus)

```
1 from typing import List, Dict, Optional, Set
2
3 def procesar_numeros(
4     numeros: List[int],
5     operacion: str = "suma"
6 ) -> int:
7     """Procesa lista de números con operación especificada"""
8     if operacion == "suma":
9         return sum(numeros)
10    elif operacion == "max":
11        return max(numeros)
12    return 0
13
14 def crear_perfil(
15     nombre: str,
16     edad: int,
17     hobbies: Optional[Set[str]] = None
18 ) -> Dict[str, any]:
19     """Crea diccionario de perfil"""
20     return {
21         "nombre": nombre,
22         "edad": edad,
23         "hobbies": hobbies or set()
24     }
```


Para profundizar

-  **Python Docs:** docs.python.org
-  **Real Python:** realpython.com
-  **PEP 8:** Guía de estilo oficial
-  **Fluent Python:** Libro avanzado
-  **Awesome Python:** Lista de recursos
-  **Corey Schafer:** YouTube tutorials
-  **LeetCode/HackerRank:** Práctica



¡La práctica hace al maestro!

¿Qué aprendimos hoy?

- ✓ Sets: colecciones únicas y operaciones matemáticas
- ✓ Diccionarios avanzados y anidados
- ✓ Funciones lambda para código conciso
- ✓ List y dict comprehensions
- ✓ *args y **kwargs para argumentos variables
- ✓ Generadores para eficiencia de memoria
- ✓ Decoradores para extender funcionalidad
- ✓ Consumo de APIs con requests
- ✓ Entornos virtuales para aislar proyectos
- ✓ Proyecto: Analizador de datos desde API



¡Preguntas!

Gracias por su atención